Nama: Muhammad Rafly Romeo Nasution
Kelas: 3KA25
NPM: 10123875

MK Praktikum Unggulan Universitas Gunadarma

Mata Kuliah: Praktikum Terapan Teori Graf (Tingkat 3)

Pertemuan V

```
!pip install anytree
from anytree import Node, RenderTree
```

```
Requirement already satisfied: anytree in /usr/local/lib/python3.12/dist-packages (2.13.0)
```

```
root = Node(10)
```

```
level_1_child_1 = Node(34, parent=root)
level_1_child_2 = Node(89, parent=root)
level_2_child_1 = Node(45, parent=level_1_child_1)
level_2_child_2 = Node(50, parent=level_1_child_2)
```

```
for pre, fill, node in RenderTree(root):
    print("%s%s" % (pre, node.name))
```

```
10
├── 34
│   └── 45
└── 89
    └── 50
```

Logika Pemrograman — Teori Graf: Struktur Pohon (Tree)

Konsep Dasar

Kode ini membangun sebuah **pohon (tree)** menggunakan library `anytree`. Pohon adalah struktur data hierarkis yang terdiri dari **node** (simpul) yang saling terhubung melalui relasi **parent-child** (induk-anak).

Breakdown Kode

1. Import Library

```
!pip install anytree
from anytree import Node, RenderTree
```

- `Node` → class untuk membuat setiap simpul pohon
- `RenderTree` → fungsi untuk merender/menampilkan pohon secara visual di terminal

2. Membuat Root Node

```
root = Node(10)
```

- Node dengan nilai `10` menjadi **akar (root)** — simpul tertinggi, tidak punya induk.

3. Membuat Node Level 1 (Anak dari Root)

```
level_1_child_1 = Node(34, parent=root)
level_1_child_2 = Node(89, parent=root)
```

- Node `34` dan `89` adalah anak langsung dari `root (10)`.
- Parameter `parent=root` menghubungkan node ke induknya.

4. Membuat Node Level 2 (Cucu dari Root)

```
level_2_child_1 = Node(45, parent=level_1_child_1)
level_2_child_2 = Node(50, parent=level_1_child_2)
```

- Node `45` adalah anak dari `34`, dan `50` adalah anak dari `89`.
- Ini membentuk **kedalaman (depth) = 2**.

5. Menampilkan Pohon

```
for pre, fill, node in RenderTree(root):
    print("%s%s" % (pre, node.name))
```

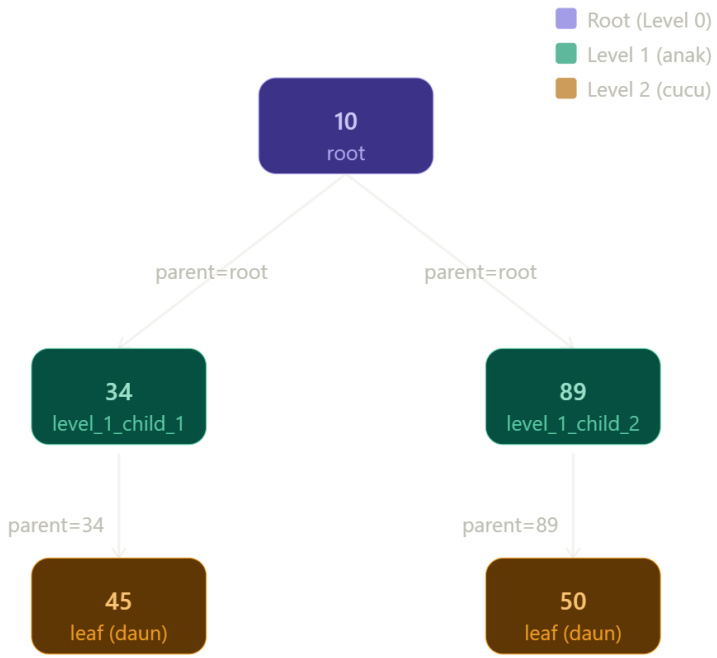
- `RenderTree(root)` menghasilkan tuple `(pre, fill, node)` untuk setiap node.
- `pre` adalah karakter prefix untuk menggambarkan struktur pohon (garis dan cabang).
- Output:

```
10
├── 34
│   └── 45
└── 89
    └── 50
```

Properti Pohon yang Terbentuk

Properti	Nilai
Root	10
Jumlah node	5
Kedalaman (depth)	2
Jumlah daun (leaf)	2 (node 45 dan 50)
Degree root	2 (punya 2 anak)

Berikut visualisasi interaktif dari pohon tersebut:



Ringkasan Alur Eksekusi

1. `Node(10)` → dibuat tanpa parent → otomatis menjadi **root**
2. `Node(34, parent=root)` & `Node(89, parent=root)` → diikat ke root → membentuk **Level 1**
3. `Node(45, parent=level_1_child_1)` & `Node(50, parent=level_1_child_2)` → diikat ke Level 1 → membentuk **Level 2 (daun/leaf)**
4. `RenderTree(root)` → melakukan **traversal DFS (Depth-First Search)** dari root ke bawah, mencetak setiap node dengan prefix visual

Node yang tidak punya anak disebut **leaf node** (daun), yaitu `45` dan `50`. Seluruh pohon ini merupakan contoh **Binary Tree** karena setiap node memiliki maksimal 2 anak.

Copyright © Pengelola MK Praktikum Unggulan (Praktikum DGX), Universitas Gunadarma

https://www.praktikum-hpc.gunadarma.ac.id/